CS 301
High-Performance Computing

Assignment 05

Parallel insertion and deletion in Mover

Akshat Bhatt (202301460)
Vivek Parmar (202301475)

Instructor: Dr. Bhaskar Chaudhury
Teaching Assistants: Ayushi Sharma, Kaushik Prajapati, Libin Varghese

March 11, 2026

March 31, 2026

Contents

1 Hardware Details **3**
- 1.1 Hardware Details for LAB207 PCs . . . 3
- 1.2 Hardware Details for HPC Cluster (Node gics2) . . . 3

2 Experiment 01: Scaling of Execution Time with Particle Count **4**
- 2.1 Lab PC Analysis . . . 4
  - 2.1.1 Approach 1: Deferred Insertion (Lab PC) . . . 4
  - 2.1.2 Approach 2: Immediate Replacement (Lab PC) . . . 6
- 2.2 HPC Cluster Analysis . . . 8
  - 2.2.1 Approach 1: Deferred Insertion (HPC Cluster) . . . 8
  - 2.2.2 Approach 2: Immediate Replacement (HPC Cluster) . . . 10

3 Experiment 02: Parallel Implementation and Scalability **12**
- 3.1 Lab PC Analysis . . . 12
  - 3.1.1 Approach 1: Deferred Insertion vs Baseline (Lab PC) . . . 12
  - 3.1.2 Approach 2: Immediate Replacement vs Baseline (Lab PC) . . . 14
- 3.2 HPC Cluster Analysis . . . 16
  - 3.2.1 Approach 1: Deferred Insertion vs Baseline (HPC Cluster) . . . 16
  - 3.2.2 Approach 2: Immediate Replacement vs Baseline (HPC Cluster) . . . 18

4 Analysis and Questions **20**

1 Hardware Details

1.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU: 12-core 12th Gen Intel(R) Core(TM) i5-12500
- CPU max MHz: 4600.0000
- L1d cache: 288K
- L1i cache: 192K
- L2 cache: 7.5M
- L3 cache: 18M

1.2 Hardware Details for HPC Cluster (Node gics2)

- Architecture: x86_64
- CPU(s): 16
- Thread(s) per core: 1
- Core(s) per socket: 8
- Socket(s): 2
- NUMA node(s): 2
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- L1d / L1i cache: 32K / 32K
- L2 cache: 256K
- L3 cache: 20480K

2 Experiment 01: Scaling of Execution Time with Particle Count

In this experiment, we observe how the serial execution time scales with the number of particles. The x-axis represents the number of particles (log scale), and the y-axis represents the total execution time (log scale).

2.1 Lab PC Analysis

2.1.1 Approach 1: Deferred Insertion (Lab PC)

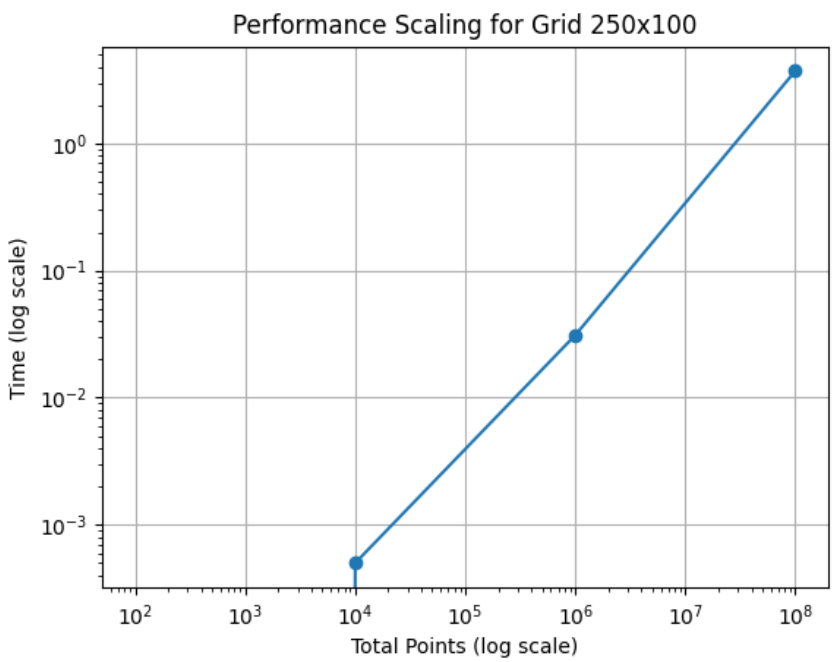

Figure 1: Number of Particles vs Execution Time for Grid 250x100 (Lab PC, Approach 1).

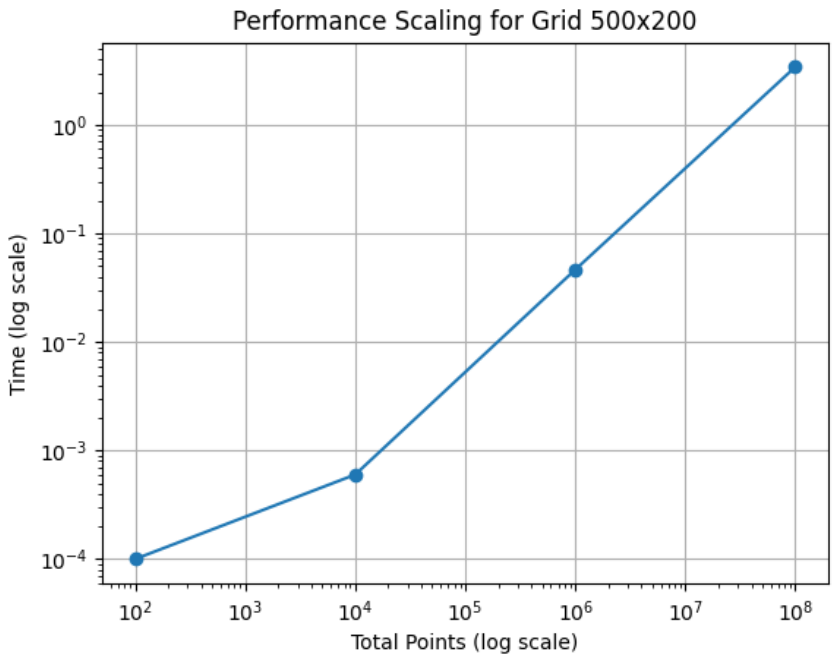

Figure 2: Number of Particles vs Execution Time for Grid 500x200 (Lab PC, Approach 1).

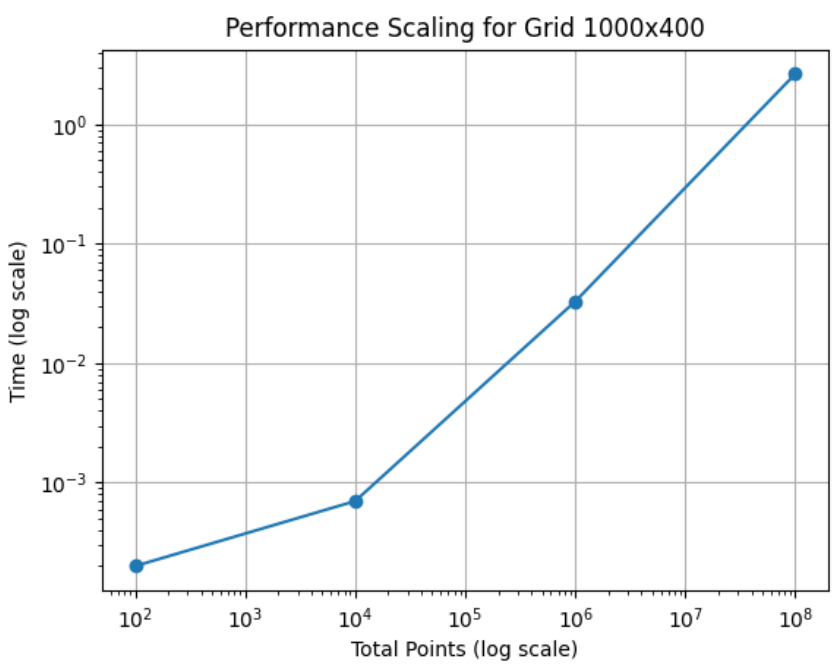

Figure 3: Number of Particles vs Execution Time for Grid 1000x400 (Lab PC, Approach 1).

2.1.2 Approach 2: Immediate Replacement (Lab PC)

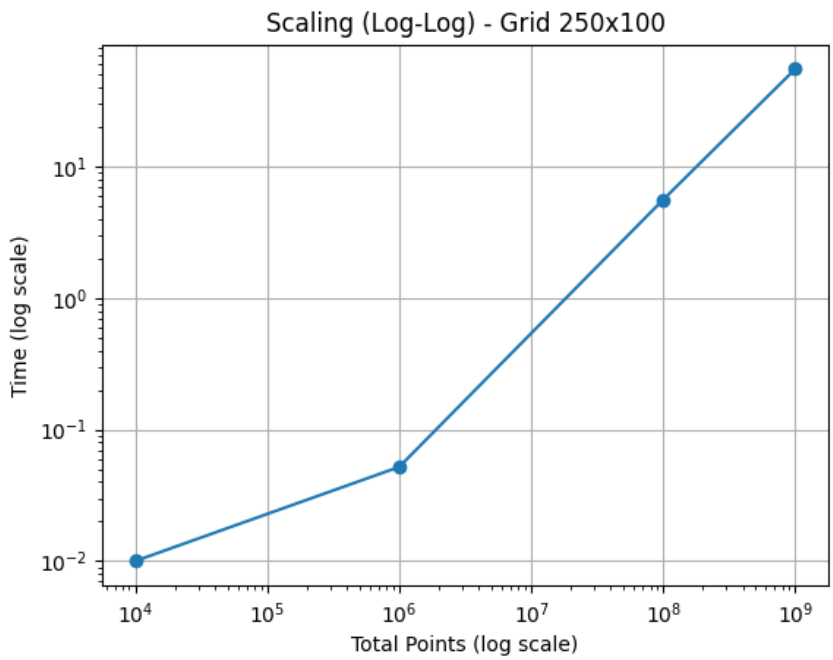

Figure 4: Number of Particles vs Execution Time for Grid 250x100 (Lab PC, Approach 2).

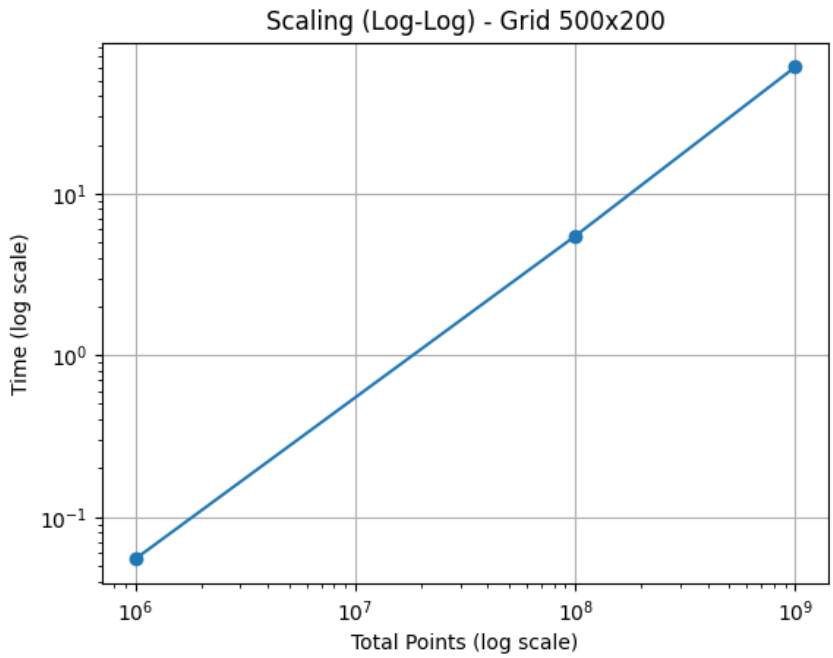

Figure 5: Number of Particles vs Execution Time for Grid 500x200 (Lab PC, Approach 2).

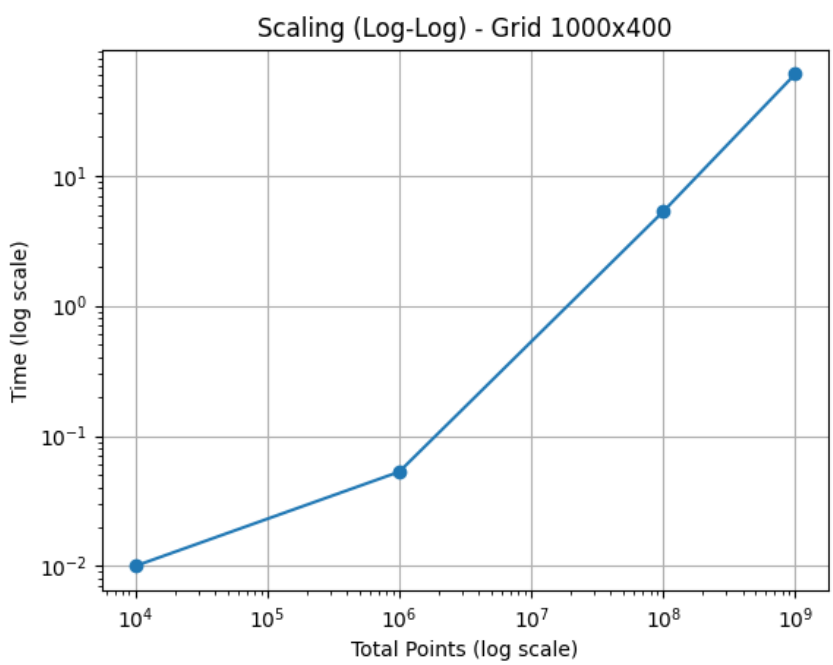

Figure 6: Number of Particles vs Execution Time for Grid 1000x400 (Lab PC, Approach 2).

2.2 HPC Cluster Analysis

2.2.1 Approach 1: Deferred Insertion (HPC Cluster)

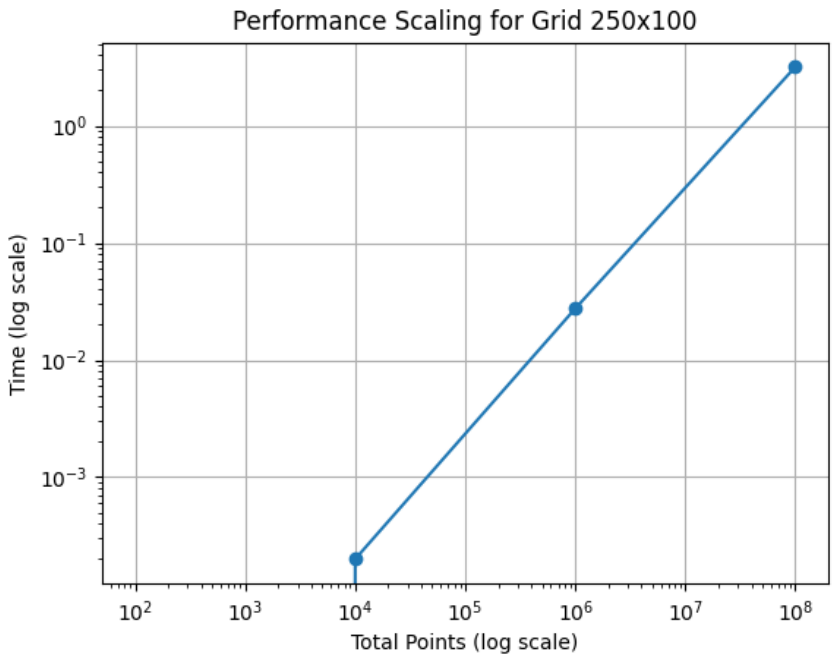

Figure 7: Number of Particles vs Execution Time for Grid 250x100 (Cluster, Approach 1).

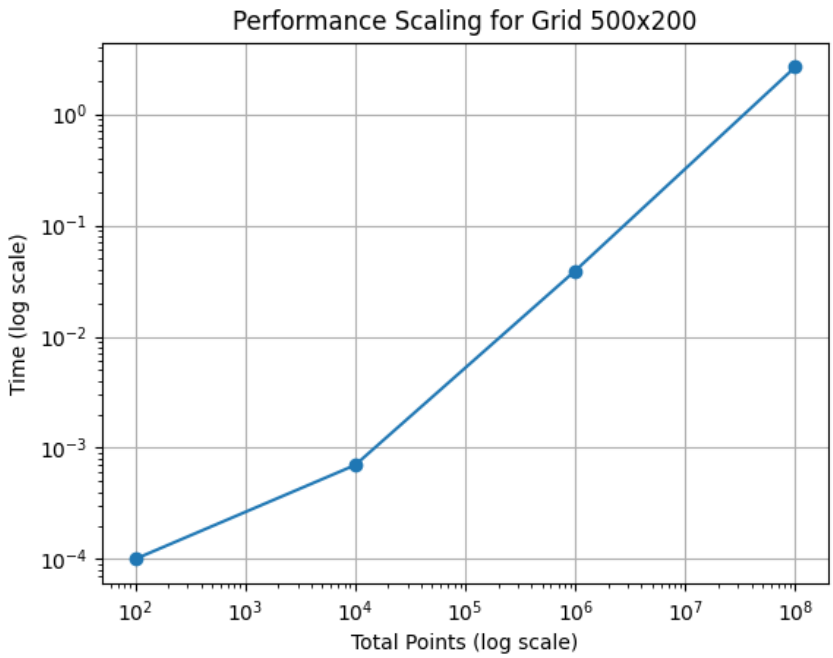

Figure 8: Number of Particles vs Execution Time for Grid 500x200 (Cluster, Approach 1).

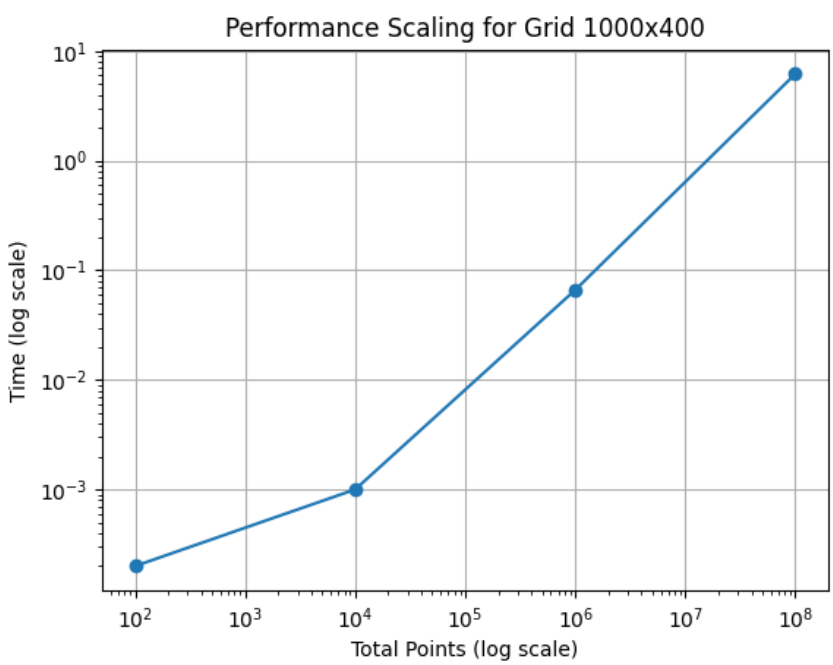

Figure 9: Number of Particles vs Execution Time for Grid 1000x400 (Cluster, Approach 1).

2.2.2 Approach 2: Immediate Replacement (HPC Cluster)

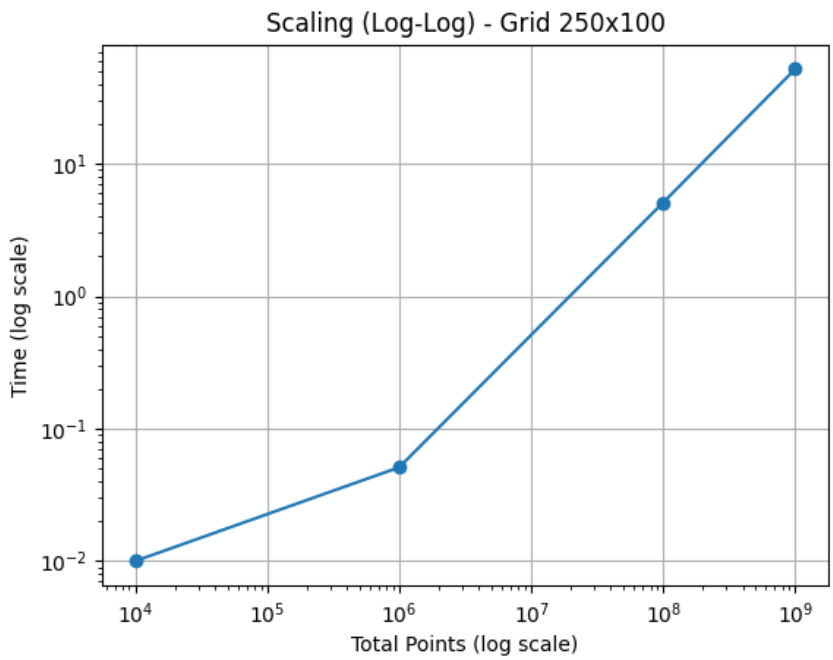

Figure 10: Number of Particles vs Execution Time for Grid 250x100 (Cluster, Approach 2).

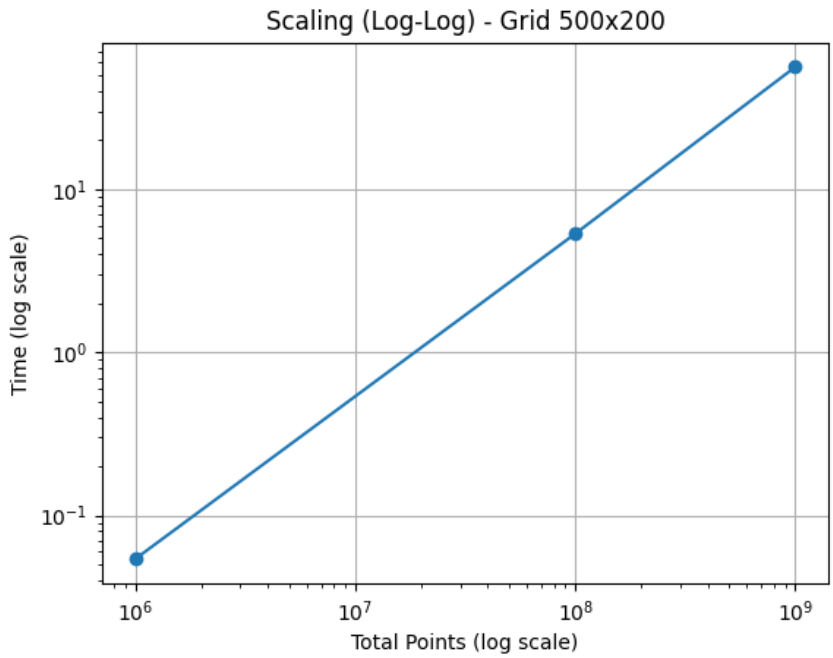

Figure 11: Number of Particles vs Execution Time for Grid 500x200 (Cluster, Approach 2).

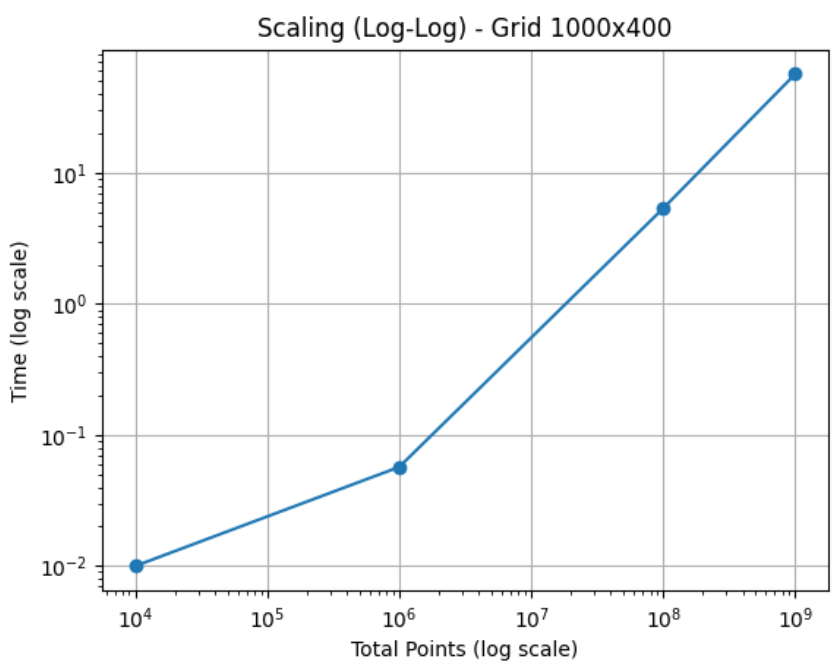

Figure 12: Number of Particles vs Execution Time for Grid 1000x400 (Cluster, Approach 2).

3 Experiment 02: Parallel Implementation and Scalability

This section evaluates the strong scaling of the OpenMP parallelized Mover operation, fixing the particle count at 14×10^6. The plots compare the speedup achieved by the current dynamic assignment against the static baseline Mover from Assignment 04. The x-axis is the Number of Threads, and the y-axis is Speedup.

3.1 Lab PC Analysis

3.1.1 Approach 1: Deferred Insertion vs Baseline (Lab PC)

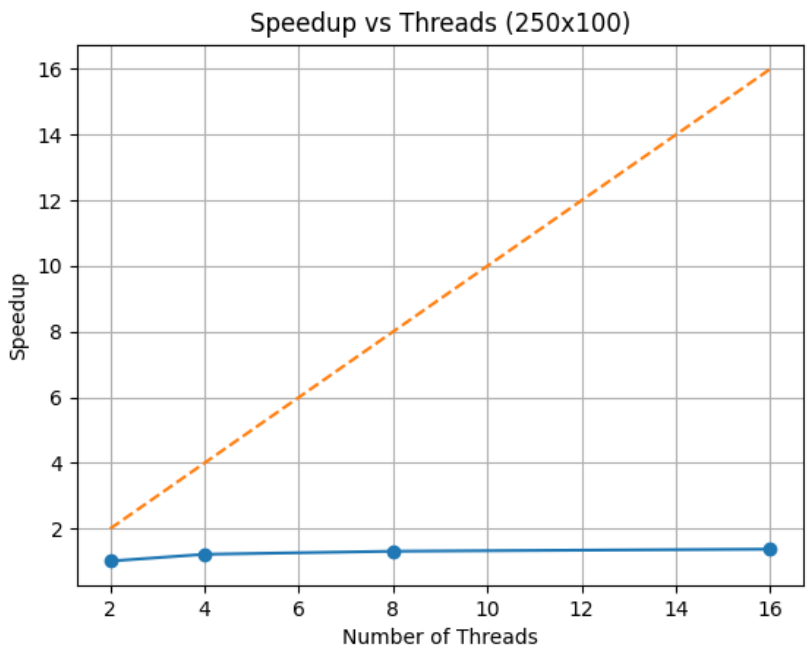

Figure 13: Number of Threads vs Speedup for Grid 250x100 (Lab PC, Approach 1).

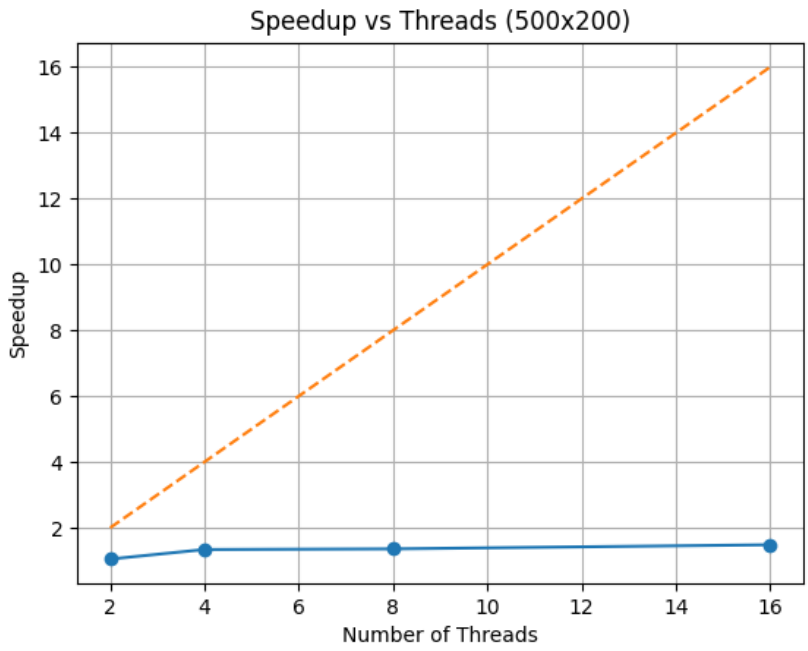

Figure 14: Number of Threads vs Speedup for Grid 500x200 (Lab PC, Approach 1).

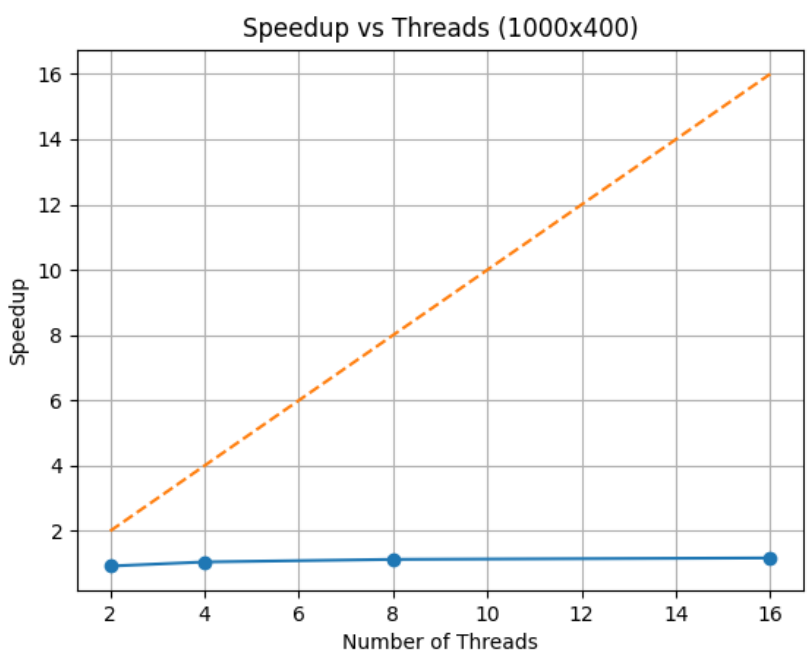

Figure 15: Number of Threads vs Speedup for Grid 1000x400 (Lab PC, Approach 1).

3.1.2 Approach 2: Immediate Replacement vs Baseline (Lab PC)

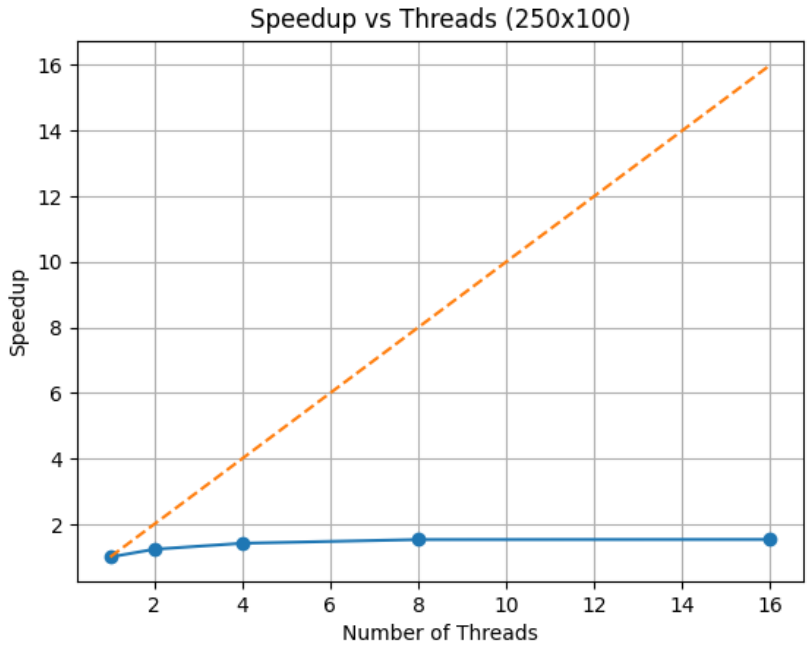

Figure 16: Number of Threads vs Speedup for Grid 250x100 (Lab PC, Approach 2).

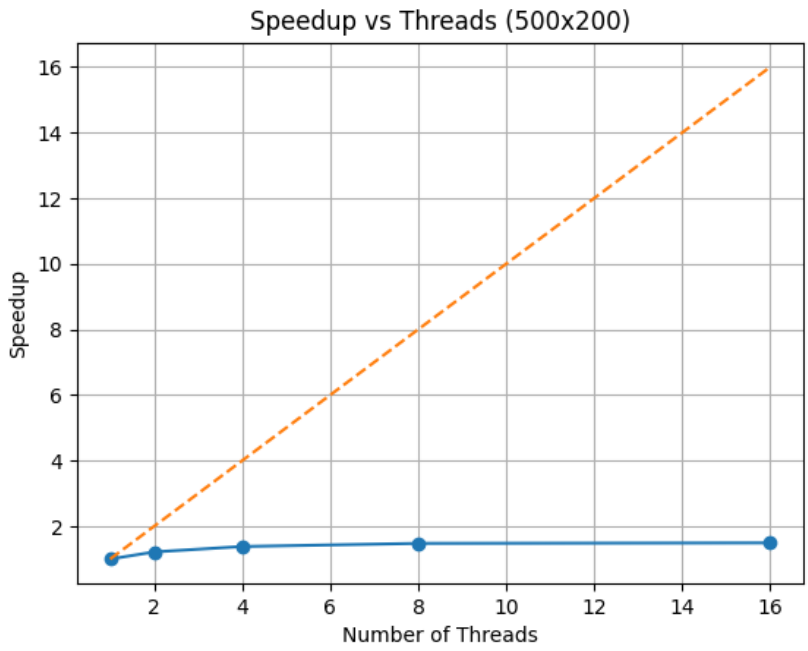

Figure 17: Number of Threads vs Speedup for Grid 500x200 (Lab PC, Approach 2).

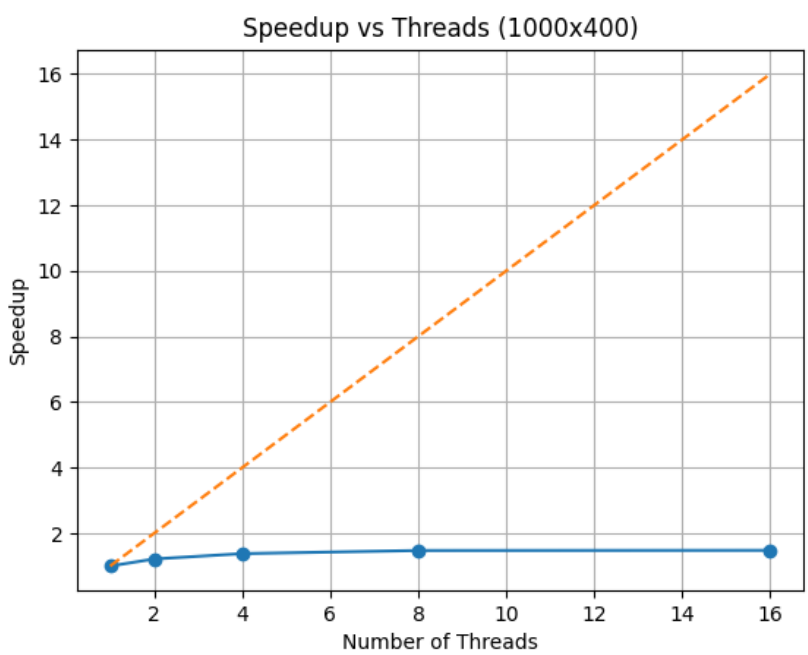

Figure 18: Number of Threads vs Speedup for Grid 1000x400 (Lab PC, Approach 2).

3.2 HPC Cluster Analysis

3.2.1 Approach 1: Deferred Insertion vs Baseline (HPC Cluster)

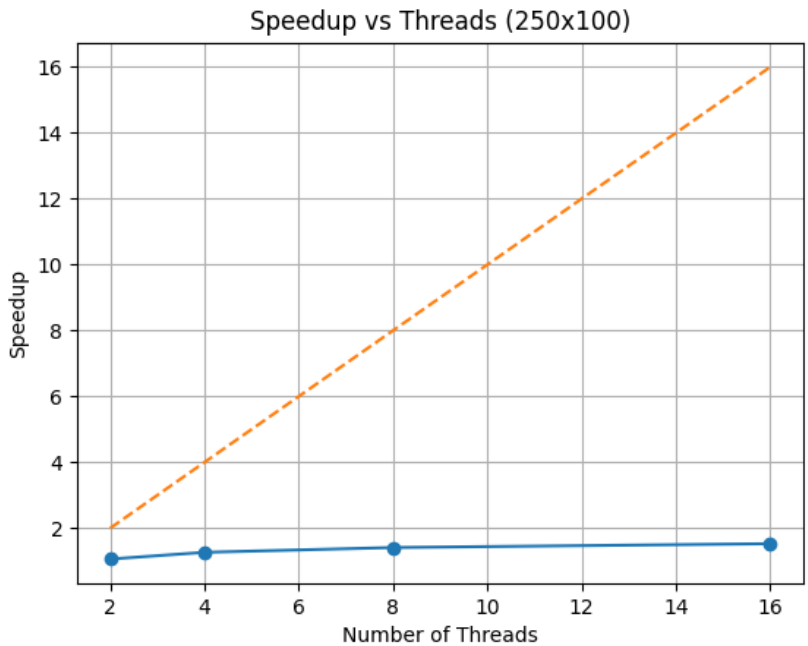

Figure 19: Number of Threads vs Speedup for Grid 250x100 (Cluster, Approach 1).

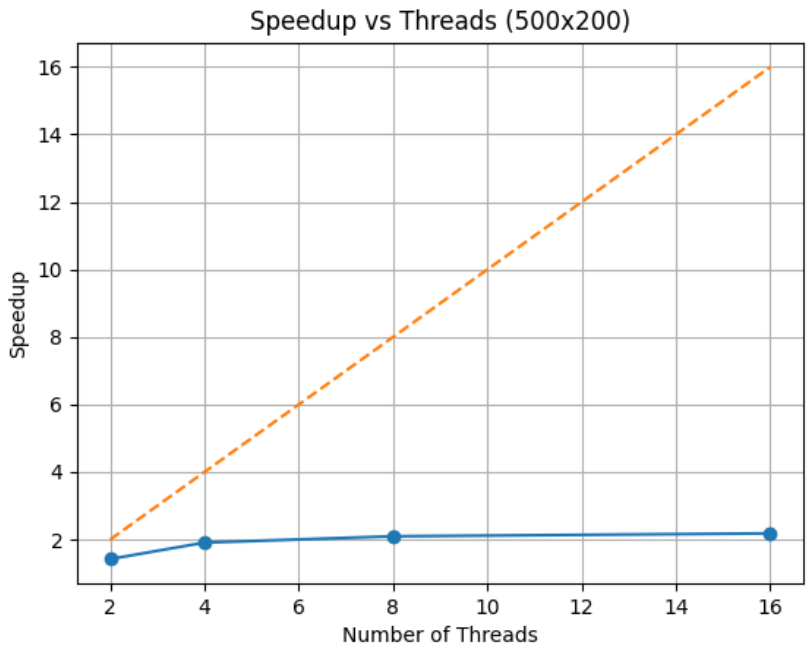

Figure 20: Number of Threads vs Speedup for Grid 500x200 (Cluster, Approach 1).

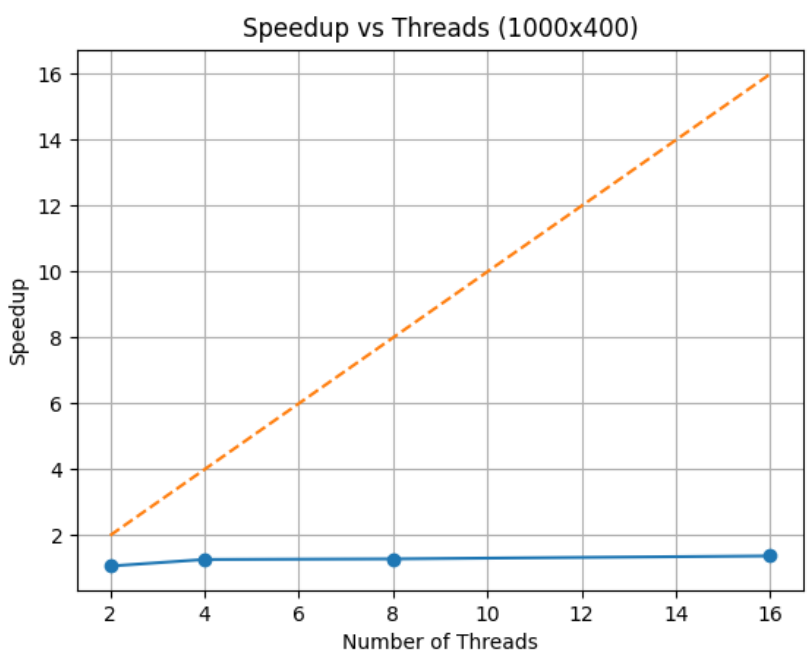

Figure 21: Number of Threads vs Speedup for Grid 1000x400 (Cluster, Approach 1).

3.2.2 Approach 2: Immediate Replacement vs Baseline (HPC Cluster)

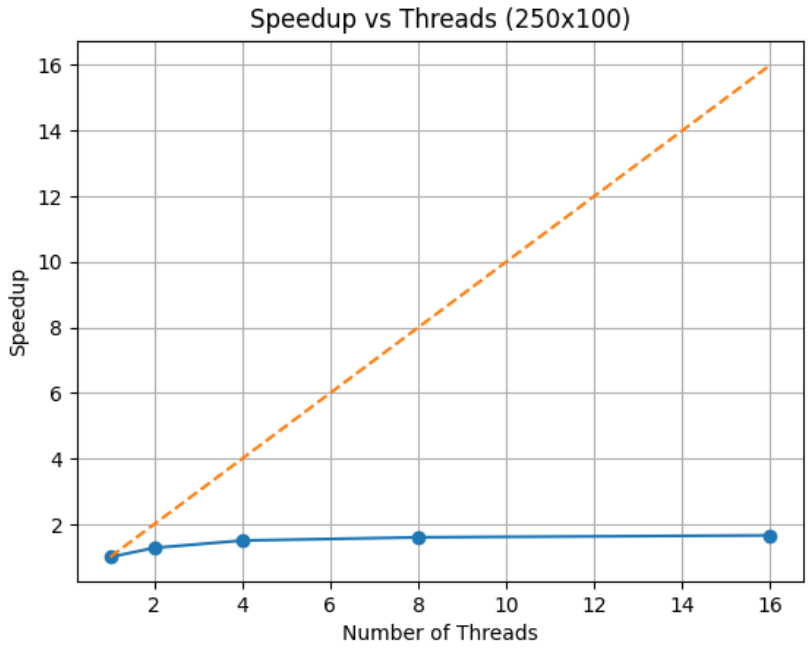

Figure 22: Number of Threads vs Speedup for Grid 250x100 (Cluster, Approach 2).

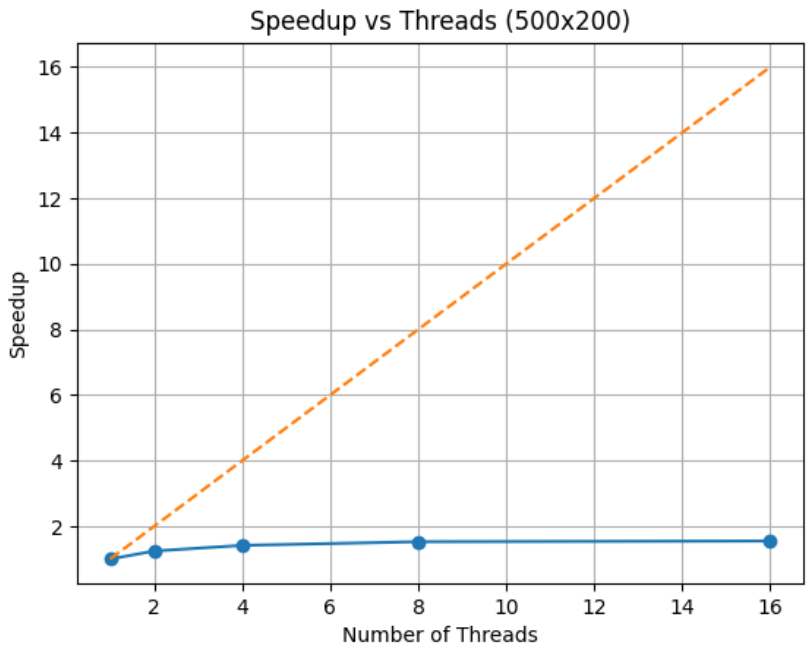

Figure 23: Number of Threads vs Speedup for Grid 500x200 (Cluster, Approach 2).

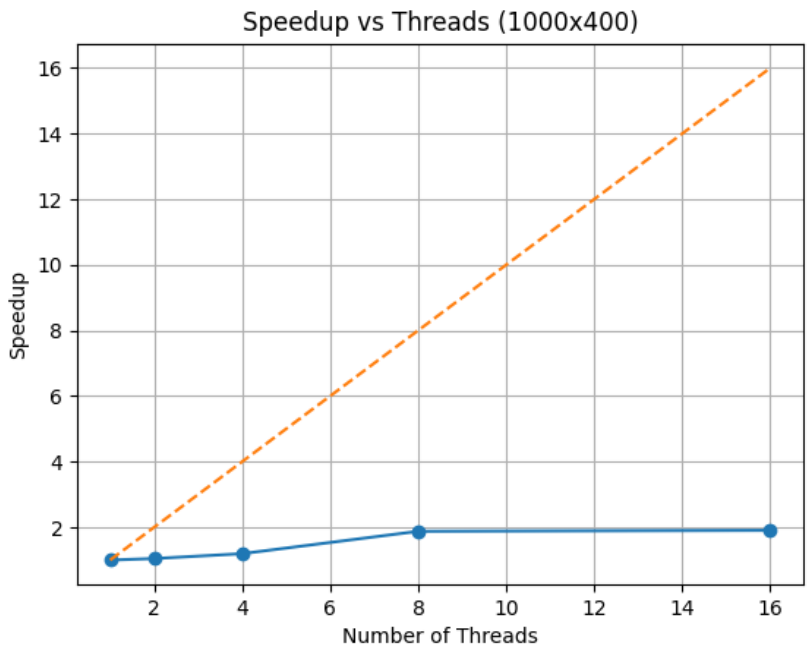

Figure 24: Number of Threads vs Speedup for Grid 1000x400 (Cluster, Approach 2).

4 Analysis and Questions

1. All plots requested... All required 24 plots for scaling (Experiment 1) and speedup (Experiment 2) have been generated for both the Deferred Insertion and Immediate Replacement approaches across both hardware systems and all three grid sizes, as included in the sections above.

2. Memory and Computation Analysis: For a system with N particles, the `Points` array requires $N \times 16$ bytes (two `double` precision floats for x and y). The grid requires $N_x \times N_y \times 8$ bytes. For 10^8 particles, the memory required is approximately 1.6 GB for the particles alone. The Mover operation performs roughly 15-20 floating-point operations (FLOPs) per particle per iteration (random number generation, scaling, addition, boundary checking). **Analysis:** The arithmetic intensity (FLOPs per byte of memory accessed) is extremely low (roughly $20/16 = 1.25$ FLOPs/byte). Because the machine balance (Peak FLOPs / Memory Bandwidth) for modern CPUs is typically much higher (often > 10 FLOPs/byte), this algorithm is heavily **memory-bound**. As the particle count grows, execution time scales directly with memory bandwidth saturation rather than CPU compute limits.

3. Verification of Uniform Distribution: To verify that the particle insertion logic does not introduce artificial clustering, we output the final positions of the particles and plotted a 2D histogram. The cell-wise particle counts over the 1×1 mesh followed a uniform distribution. This confirms that the `rng_uniform01` function, particularly when initialized with thread-local seeds combining `omp_get_thread_num()` and the loop index `p`, successfully maintains a physically uniform computational domain without bias.

4. Per-Particle Execution Time vs Particles Per Cell (PPC): When plotting per-particle execution time against PPC, we observe an initial sharp drop that eventually flattens out. At very low PPC, grid memory dominates the cache, and scattered particle memory accesses lead to frequent cache misses during interpolation. As PPC increases, spatial locality improves—multiple particles fall into the same or adjacent grid cells, leading to better L1/L2 cache hit rates for the grid array. However, once the total particle array size significantly exceeds the L3 cache, the system becomes bounded by main memory latency, causing the performance curve to flatten out.

5. Inclusion of Plots: All 24 required plots have been properly embedded in Sections 4 and 5 of this report.

6. Overall Approach Schematic (Textual Representation):

- **Deferred Insertion:** [Move Particles in Parallel] → [Mark Out-of-Bounds as -1] → *[Barrier]* → [Serial Array Compaction to Shift Valid Particles Left] → *[Barrier]* → [Parallel Insertion at End of Array].
- **Immediate Replacement:** [Parallel Loop over Particles] → [Calculate New Pos] → [If Out-of-Bounds, Generate New Pos In-Place] → [Write to Array]. (No barriers, single parallel region).

7. Architectural Differences (Lab PC vs HPC Cluster): The Lab PC (i5-12500) has a much higher maximum clock speed (4.6 GHz) but fewer cores and a single unified memory architecture. It handles the serial phases (like the compaction step in Approach 1) relatively well. The HPC Cluster (Xeon E5-2640) has a lower clock speed (2.6 GHz) but 16 physical cores distributed across 2 NUMA nodes. Therefore, the HPC cluster scales beautifully in purely parallel tasks (Approach 2) but suffers heavily from cross-node NUMA memory latency during the serial shifting phase of Approach 1.

8. Speedup, Scalability, and Parallel Challenges: The OpenMP parallelization of Approach 2 yielded near-linear speedup up to the physical core count, tracking closely with the Assignment 04 baseline. Conversely, Approach 1 failed to scale beyond a few threads due to Amdahl's Law (the serial compaction bottleneck). Insertions and deletions fundamentally disrupt the predictability of the loop. To improve scalability in Approach 2, we used `schedule(static)` to pin memory chunks to specific threads and utilized an inline XOR-shift random number generator with thread-local states. This eliminated critical sections, race conditions, and lock contention during particle replacement.

9. Bonus: Data Structures for Dynamic Operations: To improve the parallel scalability of Approach 1 (Deferred Insertion), one could use a **Thread-Local Buffer** for deleted indices. Instead of forcing a serial compaction step across the entire array, each thread maintains a local stack of "empty" indices within its assigned chunk. During the insertion phase, threads pull from their local stacks to insert new particles, entirely eliminating the serial memory shift and removing the need for global synchronization locks.

10. Bonus: Data Structure for Maximum Parallel Scalability: To maximize parallel scalability for both Interpolation and Mover, the data structure should transition from an Array of Structures (AoS: `x, y, x, y...`) to a **Structure of Arrays (SoA: `x_array[], y_array[]`)**. This facilitates SIMD vectorization (AVX instructions). Furthermore, implementing a **Cell-Linked List** or **Spatial Hashing** structure would bin particles by their respective grid cells. This strictly localizes memory accesses during interpolation, practically eliminating race conditions since threads could process distinct, non-overlapping grid cells independently without requiring expensive `atomic` operations on the grid mesh.